

# Induction and $K$ -Induction for Hyperproperties

Alberto Bombardelli<sup>1,3</sup>, César Sánchez<sup>2</sup>, and Stefano Tonetta<sup>1</sup>

<sup>1</sup> Fondazione Bruno Kessler, Trento, Italy  
`{abombardelli,tonettas}@fbk.eu`

<sup>2</sup> IMDEA Software Institute, Madrid, Spain  
`cesar.sanchez@imdea.org`

<sup>3</sup> Kyoto University, Kyoto, Japan  
`abombardelli@fos.kuis.kyoto-u.ac.jp`

## Abstract

Hyperproperties, which express system behaviors that relate multiple execution traces, cover a variety of important safety and security properties. Among these, hyperproperties of the form  $\forall\exists.\varphi$ , where  $\varphi$  is an invariant or a SafetyLTL formula, capture many practically relevant specifications including noninterference, symmetry, and robust path planning. Despite their importance, efficient verification algorithms for such properties remain limited. In this paper, we propose novel verification algorithms for such  $\forall\exists$ -hyperproperties.

Our approach extends classical induction and  $k$ -induction techniques, widely used in single-trace verification, to the hyperproperty setting. We develop hyper-inductive reasoning principles that generalize the base and step cases of induction to sets of traces, refinement techniques to strengthen the property, and incremental approaches to increase the bound of the  $k$ -induction. As a byproduct, we introduce hyper- $(k)$ -inductive invariants, as certificates to witness satisfaction of  $\forall\exists$ -hyperproperties over transition systems. We evaluate our approach on various benchmarks, demonstrating that our methods are complementary to state-of-the-art hyperproperty model checkers, and we analyze in which cases our algorithms are the most efficient.

## 1 Introduction

Hyperproperties are system properties that require relating multiple execution traces [14]. Hyperproperties are common in information-flow security [22, 36], robustness and sensitivity of cyber-physical systems [35], concurrency [9] and symmetry [20]. HyperLTL [13] is a temporal logic for hyperproperties that extends LTL with quantifiers for different traces. HyperLTL includes relational capabilities to compare different traces and temporal modalities where the temporal operators move all traces synchronously. Several algorithms and verification methods for HyperLTL have been proposed [20, 15, 25], particularly for  $\forall^*.\varphi$  hyperproperties (that is, hyperproperties without quantifier alternations).

A particularly expressive and practically useful subclass of hyperproperties are those of the form  $\forall^*\exists^*.\varphi$ , where for every trace, there exists another trace such that a property  $\varphi$  holds (which relates the corresponding states in both traces). When  $\varphi$  is restricted to *invariants* or *SafetyLTL* formulas, this class can express many important properties, including noninterference, symmetry, and robust path planning. These  $\forall\exists$ -hyperproperties are widely applicable, yet remain challenging to verify in practice. Current verification techniques include automata-based approaches [20] and bounded model checking for HyperLTL [25].

In this paper, we propose a new approach to verify  $\forall\exists$ -hyperproperties where  $\varphi$  is either an invariant or a SafetyLTL formula. Our method is based on adapting *induction* and *k-induction* techniques [33], which are widely used in trace-based verification, to the hyperproperty setting. This adaptation requires reasoning over sets of traces and generalizing inductive reasoning principles accordingly.

Our contributions are as follows:

- We define inductive and  $k$ -inductive proof rules for  $\forall\exists$ -hyperproperties, extending base and step reasoning to trace sets. As a byproduct, we introduce hyper- $(k)$ -inductive invariants, as certificates to witness satisfaction of  $\forall\exists$ -hyperproperties over transition systems.
- We define a refinement principle that strengthens the given  $\forall\exists$  specification to make it inductive, so that the property can be proved using our generalized rules.
- We introduce incremental strategies to increase the bound of  $k$ -induction learning from previous checks.
- We implement our approach and evaluate it on a range of benchmarks from security, distributed systems, and motion planning. Our results show that our technique is complementary to existing model checkers for HyperLTL, and often scales better for specifications of the  $\forall\exists$  type.

**Outline.** The rest of the paper is organized as follows. Section 1 discusses related work. In Section 2, we review preliminaries on hyperproperties, induction and  $k$ -induction. Then, Section 3 introduces our generalization of induction and  $k$ -induction to the hyper setting, and describes refinement and incremental techniques. Section 4 presents our implementation and experimental evaluation. Finally, in Section 5 we draw some conclusions and directions for future work.

**Related Work.** The HyperLTL model checking problem has been studied following various approaches. In [13, 20] the authors define an automata-based construction for HyperLTL. For the quantifier-alternation free fragment of HyperLTL, this approach reduces to self-composition of the system [1] followed by LTL model checking. The self-composition approach has been successfully implemented in MCHyper [20], which reduces the verification problem to symbolic model checking. In contrast, AutoHyper [6] implements the automata-based construction using an explicit state approach via language inclusion using inclusion checkers such as Spot [17]. AutoHyper can handle any type of HyperLTL formula, independently of the amount of quantifier alternations or temporal nature of the temporal sub-formula. However, due to its explicit state nature, AutoHyper does not scale very well with the size of the system.

Another line of work for verifying HyperLTL was introduced by Coenen et al. [15], who proposed a game-based approach for proving  $\forall\exists$  properties using strategies. In this technique, verification is framed as a game between a universal trace and an existential trace. The strategy for the existential trace can either be provided by the user or synthesized automatically, which in [15] is searched using a bounded approach. Because the existential trace is allowed to observe the entire execution of the universal trace, the method requires the use of prophecy variables to anticipate future behaviors. This approach was implemented in an extended version of MCHyper. Subsequently, [4] shows that completeness of this technique can only be guaranteed when using  $\omega$ -regular prophecy variables. The authors provide a constructive proof of this result and implemented the improved technique in the tool HyPro [4]. The approach has also been applied to asynchronous hyperproperties in [5], where predicate abstraction was used to handle infinite-state systems.

The game-based verification method was adapted to synchronize traces at designated observation points, enabling reasoning over asynchronous executions. These approaches have similarities with our technique because they search a strategy to prove the property. However, our technique does not search directly a strategy via prophecies, but we refine counter-examples to induction and look for  $k$ -inductive  $\forall\exists$  invariants. Recently, [27] transformed the game-based construction of [5] to a CHC satisfiability problem, supporting that the CHC transformation

outperforms the original game-based approach of [5]. Extending our approach using predicate abstraction and supporting asynchronous hyperproperties is future work.

In [25], the authors introduce a Bounded Model Checking (BMC) technique for HyperLTL, extending the classic BMC approach [8] to support quantified formulas using QBF solvers. Unlike the original BMC framework, this method does not support looping conditions and instead adopts a finite semantics akin to the truncated semantics used in LTL [19]. Later, [24] extended BMC for the asynchronous Hyperlogic A-HLTL [2]. Also, [26] focuses on proving validity of  $\forall\exists$ -safety and  $\exists\forall$ -safety properties through inductive reasoning. Their approach resembles our basic inductive algorithm, with the key distinction that the approach in [26] instantiates quantifiers and relies on SAT solvers to verify the property. In contrast, our method provides completeness guarantees and incorporates invariant strengthening as part of the proof process.

Besides the works on hyperproperties, the main inspiration for our work is  $k$ -induction [33], which extends classical Bounded Model Checking [8] with an inductive reasoning to prove properties. This technique is widely used in industry [31] and has been extended in various ways, such as using incrementality [18], predicate abstraction [34], abstract interpretation [11], and parallel computing [28]. As far as we know, this paper is the first work lifting this technique from trace properties to hyperproperties.

## 2 Preliminaries

**Symbolic Transition System.** We assume that systems have finite states and are described with Boolean variables and propositional formulas, although most of the results (except for completeness) can be lifted to the infinite-state systems.

A *Symbolic Transition System* (STS)  $M$  is a tuple  $M = \langle V, I, T \rangle$  where  $V$  is a set of (state) variables,  $I(V)$  is a formula representing the initial states, and  $T(V, V')$  is a formula representing the transitions. A *state* of  $M$  is an assignment to the variables  $V$ . A (finite) *path* of  $M$  is an infinite sequence  $s_0, s_1, \dots$  (resp., finite sequence  $s_0, s_1, \dots, s_k$ ) of states such that  $s_0 \models I$  and, for all  $i \geq 0$  (resp.,  $0 \leq i < k$ ),  $s_i, s'_{i+1} \models T$ . Given two transition systems  $M_1 = \langle V_1, I_1, T_1 \rangle$  and  $M_2 = \langle V_2, I_2, T_2 \rangle$ , we denote by  $M_1 \times M_2$  the synchronous product  $\langle V_1 \cup V_2, I_1 \wedge I_2, T_1 \wedge T_2 \rangle$ .

A state property of  $M = \langle V, I, T \rangle$  is specified over a set  $V' \subseteq V$  of variables. We use  $\text{AP}$  for the set of state properties. The alphabet is therefore given by the set of assignments to  $V'$ . A finite or infinite path defines a corresponding trace given by restricting the assignments to  $V'$ . We use  $\text{Traces}(M)$  for the set of traces of  $M$ .

**LTL and SafetyLTL.** We consider LTL [32], which has the following syntax:

$$\varphi ::= \top \mid a \mid \varphi \vee \varphi \mid \neg \varphi \mid \mathbf{X}\varphi \mid \varphi \mathbf{U} \varphi$$

where  $a$  is drawn from  $\text{AP}$ ,  $\top$  is the true constant,  $\vee$  and  $\neg$  are the usual Boolean disjunction and negation, while  $\mathbf{X}$  and  $\mathbf{U}$  are respectively the “next” and “until” temporal operators. The semantics of LTL associates pointed traces with formulae as follows (we omit Boolean connectives, which are standard):

$$\begin{aligned} (\sigma, i) \models a & \iff a \in \sigma(i) \\ (\sigma, i) \models \mathbf{X}\varphi & \iff i < |\sigma| - 1 \text{ and } (\sigma, i+1) \models \varphi \\ (\sigma, i) \models \mathbf{Y}\varphi & \iff i > 0 \text{ and } (\sigma, i-1) \models \varphi \\ (\sigma, i) \models \varphi_1 \mathbf{U} \varphi_2 & \iff \text{for some } j \geq i \text{ } (\sigma, j) \models \varphi_2, \text{ and for all } k \in [i, j), (\sigma, k) \models \varphi_1 \end{aligned}$$

We also use common derived operators like  $\wedge$ ,  $\mathbf{R}$ ,  $\mathbf{F}$ ,  $\mathbf{G}$ ,  $\mathbf{Z}$  (dual of  $\mathbf{Y}$ ) and  $\mathbf{T}$  (dual of  $\mathbf{S}$ ). SafetyLTL, the safety fragment of LTL, is obtained by disallowing positive occurrences of Until in the formula. We define this fragment in negative normal form as follows:

$$\varphi_S ::= \top \mid \perp \mid \neg a \mid a \mid \varphi_S \vee \varphi_S \mid \varphi_S \wedge \varphi_S \mid \mathbf{X}\varphi_S \mid \varphi_S \mathbf{R} \varphi_S$$

**The Temporal Logic HyperLTL.** HyperLTL [13] is a temporal logic for hyperproperties. The syntax of HyperLTL is:

$$\alpha ::= \exists x.\alpha \mid \forall x.\alpha \mid \varphi \qquad \varphi ::= a[x] \mid \varphi \vee \varphi \mid \neg\varphi \mid \mathbf{X}\varphi \mid \varphi \mathbf{U} \varphi$$

where  $x$  is a *trace variable* from an infinite set  $\text{VAR}$  of trace variables. The intended meaning of  $a[x]$  is that state property  $a \in \text{AP}$  holds at the current time in trace  $x$ . Trace quantifiers  $\exists x$  and  $\forall x$  allow reasoning simultaneously about different traces of the computation. Atomic predicates  $a[x]$  refer to a single trace  $x$ . Given an LTL formula  $\alpha$ , we denote by  $\alpha[x]$  the formula obtained by substituting every atomic proposition  $a$  with  $a[x]$ .

Given a HyperLTL formula  $\varphi$ , we use  $\text{TrVars}(\varphi)$  for the set of trace variables quantified in  $\varphi$ . A formula  $\varphi$  is well-formed if for all atoms  $a[x]$ ,  $x$  is quantified in  $\varphi$ . Given a set of infinite traces  $T$ , the semantics of a HyperLTL formula  $\alpha$  is defined in terms of *pointed trace assignments*, which are partial mappings of the form  $\Pi : \text{TrVars}(\alpha) \rightarrow (T \times \mathbb{N})$ . We use  $\text{Dom}(\Pi)$  for the subset of  $\text{TrVars}(\varphi)$  for which  $\Pi$  is defined. Given a trace assignment  $\Pi$ , a trace variable  $x$ , a trace  $\sigma$  and a pointer  $p$ , we denote by  $\Pi[x \mapsto (\sigma, p)]$  the assignment that coincides with  $\Pi$  for every trace variable except for  $x$ , which is mapped to  $(\sigma, p)$ . The trace assignment with empty domain is denoted by  $\Pi_0$ . Also, we use  $\Pi + n$  to denote trace assignment  $\Pi'$  such that  $\Pi'(x) = \Pi(x) + n$  for all  $x \in \text{Dom}(\Pi) = \text{Dom}(\Pi')$ . Given a partial trace assignment  $\Pi$  and a set of traces  $T$ , the semantics of HyperLTL is as follows (we again omit  $\vee$  and  $\neg$ ):

$$\begin{aligned} \Pi &\models_T \exists x.\alpha && \Leftrightarrow \text{for some } \sigma \in T, \Pi[x \mapsto (\sigma, 0)] \models_T \alpha \\ \Pi &\models_T \forall x.\alpha && \Leftrightarrow \text{for all } \sigma \in T, \Pi[x \mapsto (\sigma, 0)] \models_T \alpha \\ \Pi &\models_T \varphi && \Leftrightarrow \Pi \models \varphi \\ \Pi &\models a[x] && \Leftrightarrow a \in \sigma(p), \text{ where } (\sigma, p) = \Pi(x) \\ \Pi &\models \varphi_1 \mathbf{U} \varphi_2 && \Leftrightarrow \text{for some } j \geq 0 \text{ } (\Pi + j) \models \varphi_2, \text{ and for all } i < j, (\Pi + i) \models \varphi_1 \\ \Pi &\models \mathbf{X}\varphi && \Leftrightarrow (\Pi + 1) \models \varphi \end{aligned}$$

Note that quantifiers assign traces to trace variables and set the pointer to the initial position 0. We say that a set of infinite traces  $T$  is a model of a HyperLTL formula  $\varphi$ , denoted  $\Pi \models \varphi$  whenever  $\Pi_0 \models_T \varphi$ . We say that an STS  $M$  is a model of a HyperLTL formula  $\varphi$ , denoted by  $M \models \varphi$ , whenever  $\text{Traces}(M) \models \varphi$ .

**Example 1.** We now show an example with an STS satisfying an HyperLTL temporal-safety property: Let  $M_{ex} := \langle \{ch, o, h, p\}, \neg ch, (ch' \leftrightarrow \neg ch) \wedge (ch \rightarrow (o' \leftrightarrow (h \leftrightarrow p))) \rangle$ .  $M_{ex}$  represents a system with 2 “modalities”, one in which it chooses a value for  $o$  ( $ch = \top$ ) depending on  $h$  and  $p$ , and the other where  $o$  can take any possible value. The forall-exists Hyperproperty that we are interested in proving is  $\varphi_{ex} := \forall x \exists y. \mathbf{G}((o[x] \leftrightarrow o[y]) \wedge (h[x] \leftrightarrow \neg h[y]))$ . The property states that, for every execution producing an output value ( $o$ ), there exists another execution with the same value of ( $o$ ) but a different value of the secret variable ( $h$ ). In the rest of the paper, we use this example, with minor variations, to illustrate the algorithms.

**Bounded Model Checking.** Bounded Model Checking (BMC) [8] considers only initial paths of length up to a certain bound. Given a bound  $k$ , an STS  $M$  and a formula  $\varphi(V)$ , the

bounded reachability problem is the problem of finding, for some  $j \leq k$  an initial finite path  $s_0, \dots, s_j$  of  $M$  such that  $s_j \models \varphi$ . In BMC, the problem is usually reduced to a satisfiability problem.

In the following, given a set  $V$  of variables, we use several copies of  $V$ , one for each state of the path we are looking for. We will denote with  $V_i$  the  $i$ -th copy of  $V$ . Thus,  $V_i = \{v_i\}_{v \in V}$ , where  $v_i$  represents the  $i$ -th copy of  $v$ . For a given bound  $k$ , we often use  $k+1$  copies  $V_0, \dots, V_k$  and, when  $k$  is clear from the context, we denote their union by  $\bar{V}$ .

Given an STS  $M$ , and a bound  $k$ , the formula  $Unroll_k(\bar{V})$  is defined as follows:

$$Unroll_k(\bar{V}) := \bigwedge_{1 \leq h \leq k} T(V_{h-1}, V_h)$$

Given an STS  $M$ , a bound  $k$ , and a formula  $\varphi$ , the formula  $BMC_k(\bar{V})$  is defined as follows:

$$BMC_k(\bar{V}) := I(V_0) \wedge Unroll_k(\bar{V}) \wedge \bigvee_{0 \leq i \leq k} \varphi(V_i)$$

The formula  $BMC_k$  encodes the bounded reachability problem, in the sense that  $BMC_k$  is satisfiable if and only if there exists an initial path of length up to  $k$  reaching  $\varphi$ .

**$K$ -induction**  $K$ -induction [33] is a technique that proves that if a set of states is not reachable in  $k$  steps, then it is not reachable at all. Analogous to the induction principle,  $k$ -induction consists of a *base step*—which solves the bounded reachability problem with a given bound  $k$  of steps—and an *inductive step*, which concludes that  $k$  is sufficient to solve the (unbounded) reachability problem. The idea of the inductive step is to check whether the target set of states cannot be reached in  $k$  steps. Both of these checks can be solved by means of satisfiability.

Given an STS  $M$ , and a bound  $k$  and a formula  $\varphi$ , the formula  $KIND_k$  is defined as follows:

$$KIND_k := Unroll_k(\bar{V}) \wedge \bigwedge_{0 \leq i < j \leq k} \neg \bigwedge_{v \in V} v_i = v_j \wedge \bigwedge_{0 \leq i < k} \neg \varphi(V_i) \wedge \varphi(V_k)$$

If  $KIND_k$  is unsatisfiable, then  $M$  does not have a uninitialized simple path (i.e., a path without repetitions) reaching  $\varphi$  in  $k$  steps.

If  $BMC_k$  and  $KIND_k$  are both unsatisfiable, then  $\varphi$  is not reachable in  $M$ . Thus, in order to prove that  $\varphi$  is not reachable one can check the unsatisfiability of  $BMC_k$  and  $KIND_k$  for increasing values of  $k$ . Usually, these checks can enjoy sharing lemmas discovered trying to prove unsatisfiability, and therefore the checks are performed using incremental SAT solvers.

### 3 Induction and $K$ -Induction for Hyperproperties

Induction and  $k$ -induction are two well-known techniques that are used to prove symbolically invariant (trace) properties. In this section we present two novel techniques constructed adapting induction,  $K$ -induction and the notion of generalization of counter-examples to induction for hyperproperties. We showcase our techniques with examples and we demonstrate their soundness and completeness. This Section is organized as follows. Section 3.1 presents Induction adapted for Hyperproperties showing soundness of the algorithm; Section 3.2 presents  $k$ -induction adapted for Hyperproperties showing soundness and completeness; Section 3.3 presents refinement techniques to generalize counter-example to induction for our algorithms;

**Algorithm 1:** Induction algorithm

---

**Input:** A STS  $M = \langle V, I, T \rangle$  and a formula  $\varphi := \forall x. \exists y. \mathbf{G}\psi$

- 1 **if**  $BASE^{\forall\exists}$  *does not hold* **then return** INVALID
- 2 **else if**  $IND^{\forall\exists}$  *holds* **then return** VALID
- 3 **else return** UNK

---

finally, Section 3.4 presents an incremental encoding of the k-induction algorithm to be used with incremental solvers.

In the rest of the paper, we assume that  $M$  is a deadlock-free STS, and that the property under analysis is of the form  $\varphi := \forall x. \exists y. \mathbf{G}\psi$  where  $\psi$  is a state predicate over  $V[x]$  and  $V[y]$  and  $P := \psi$ .

For any trace variable  $z \in \text{VAR}$ , we denote  $V[z] := \{v[z] \mid v \in V\}$ ,  $I^z := I(V[z])$ ,  $T^z := T(V[z], V'[z])$ ; moreover, for a given bound  $k$ , we consider  $k+1$  variable set copies  $V[z]_0, \dots, V[z]_k$  with their union denoted as  $V[z]_0^k$ . Finally, for any  $i$ , we denote  $P_i := P(V[x]_i, V[y]_i)$ , for any  $z \in \text{VAR}$ :  $T_i^z := T(V[z]_{i-1}, V[z]_i)$ .

Although the theorems presented in this section are introduced for  $\forall x. \exists y. \mathbf{G}\psi$  properties, all proofs extended straightforwardly to the  $\forall^* \exists^*. \mathbf{G}\psi$  case by applying self-composition.

### 3.1 Induction

The induction principle for proving an invariant trace property  $\psi$  consists of finding an auxiliary property  $P$  such that the following hold: (i)  $P$  holds at all initial states; (ii) anytime that  $P$  holds, each successor state also satisfies  $P$ ; (iii)  $P$  implies the original invariant property  $\psi$ . Finally, *counter-example to induction* (CTI) is then a pair of assignments  $(X^{ce}, X^{ce'})$  over  $V, V'$  such that  $P(X^{ce})$  and  $T(X^{ce}, X^{ce'})$  holds but  $P(X^{ce'})$  is false. The adaptation of induction for hyperproperties must take into account quantifier alternation and the self-product of the system. The base case  $BASE^{\forall\exists}$  is defined as follows:

$$BASE^{\forall\exists} := \forall V[x]. \exists V[y]. I^x \rightarrow (I^y \wedge P(V[x], V[y])).$$

$BASE^{\forall\exists}$  checks whether for each initial state of  $M$  (instantiated with  $x$  trace variable), there is an initial state of  $M$  (instantiated with  $y$  trace variable) such that  $P(V[x], V[y])$  holds. We define the inductive case  $IND^{\forall\exists}$  as follows:

$$IND^{\forall\exists} := \forall V[x]V[x']V[y]. \exists V[y'] . T^x \wedge P(V[x], V[y]) \rightarrow T^y \wedge P(V[x'], V[y']).$$

$IND^{\forall\exists}$  universally quantifies over states of the self-product of  $M$  that satisfy  $P$ , and checks that for each successor of  $x$  there is a successor for  $y$  such that the property still holds. Interestingly, this property is much stronger than the target  $\forall x. \exists y. \mathbf{G}\psi$  property because we are requiring each state of the self-product to satisfy this property.

These proof obligations are combined to obtain Algorithm 1 to verify  $\forall x \exists y. \mathbf{G}\psi$ .

**Theorem 1.** *If  $\not\models BASE^{\forall\exists}$  then  $M \not\models \forall x \exists y. \mathbf{G}\psi$ . Moreover, If  $\models BASE^{\forall\exists}$  and  $\models IND^{\forall\exists}$  then  $M \models \forall x \exists y. \mathbf{G}\psi$ . Consequently, Algorithm 1 is sound.*

**Example 2.** *Consider a variation of Ex. 1, where we highlight the new parts:*

$$\varphi_{ex1} := \forall x \exists y. \mathbf{G}((o[x] \leftrightarrow o[y]) \wedge (h[x] \leftrightarrow \neg h[y]) \wedge (p[x] \leftrightarrow \neg p[y]) \wedge (ch[x] \leftrightarrow ch[y]))$$

This example property is satisfied by  $M_{ex}$ . The variable  $ch$  is true only at odd positions (counting from 0) of the trace and, choosing  $p[y]$  as the negation of  $p[x]$  ensures that  $o[y]$  is set to the same value of  $o[x]$  when  $ch$  is true. The proof obligations instantiated for the properties are as:

$$BASE^{\forall\exists} : \forall V_{ex}[x] \exists V_{ex}[y]. \neg ch[x] \rightarrow \neg ch[y] \wedge o[x] = o[y] \wedge h[x] = \neg h[y] \wedge p[x] = \neg p[y] \wedge ch[x] = ch[y]$$

$$IND^{\forall\exists} : \forall V_{ex}[x] V_{ex}[x]' V_{ex}[y] \exists V_{ex}[y]'. \left( \begin{array}{l} o[x] = o[y] \wedge h[x] = \neg h[y] \wedge (p[x] = \neg p[y]) \wedge \\ (ch[x] = ch[y]) \wedge T(V_{ex}[x], V_{ex}[x]') \end{array} \right) \rightarrow \left( \begin{array}{l} o[x]' = o[y]' \wedge h[x]' = \neg h[y]' \wedge p[x]' = \neg p[y]' \wedge \\ ch[x]' = ch[y]' \wedge T(V_{ex}[y], V_{ex}[y]') \end{array} \right)$$

$BASE^{\forall\exists}$  is valid because it is sufficient for  $y$  to choose  $\neg ch$  and then simply assign the remaining variables satisfying  $\varphi_{ex1}$ .  $IND^{\forall\exists}$  is valid as well. If  $ch[x]$  is false, then it is possible to choose  $V_{ex}[y]'$  satisfying  $\varphi_{ex1}$ . Otherwise, since  $p[y] = \neg p[x]$  and  $h[y] = h[x]$  then  $h[y] = p[y]$  iff  $h[x] = p[x]$  which means that  $o[x]' = o[y]'$ . It should be noted that if we remove  $p[x] \leftrightarrow \neg p[y]$  from  $\varphi_{ex1}$ , the property (denoted as  $\varphi'_{ex1}$ ) is no longer inductive and  $IND^{\forall\exists}$  is not satisfied. A counterexample to induction for that weakened property would be any assignment over  $V_{ex}[x], V_{ex}[x]'$  and  $V_{ex}[y]$  such that  $ch[x] = ch[y] = \top$  and  $p[x] = p[y]$ .

### 3.2 $K$ -Induction for Hyperproperties

$K$ -induction, a well-known technique for the verification of invariant trace properties, extends induction by reasoning over paths of length  $k$ . The base case of the  $k$ -induction algorithm is bounded model checking (BMC) while the inductive case takes the unrolling of the transition system of length  $k$  in conjunction with the property for  $k$  steps to entail the property at step  $k + 1$ . We adapt here the proof obligations for  $k$ -induction to hyperproperties.

The BMC proof obligation is a special case of the more generic encoding introduced in [25].

$$BMC_k^{\forall\exists} := \forall V[x]_0^k \exists V[y]_0^k. I(V[x]_0) \wedge Unroll_k(\overline{V[x]}) \rightarrow I(V[y]_0) \wedge Unroll_k(\overline{V[y]}) \wedge \bigwedge_{0 \leq i \leq k} P_i$$

**Lemma 1.** *There is a  $k \geq 0$  such that  $BMC_k^{\forall\exists}$  does not hold iff  $M \not\models \forall x \exists y. \mathbf{G}\psi$*

The proof obligation  $KIND_k^{\forall\exists}$  extends  $IND^{\forall\exists}$  with the universally quantified unrolling of the self-product of  $M$  in conjunction of the property at each position  $i < k$ :

$$KIND_k^{\forall\exists} := \forall V[x]_0^k \forall V[y]_0^{k-1} \exists V[y]_k. Unroll_k(\overline{V[x]}) \wedge Unroll_{k-1}(\overline{V[y]}) \wedge \bigwedge_{0 \leq i < k} P_i \rightarrow T_k^y \wedge P_k$$

**Example 3.** Consider the following variation of Ex. 1, which models a finite program with a bounded counter  $c$  ranging from 0 to some  $j \in \mathbb{N}$ . After exactly  $j$  steps, the output  $o$  is set to  $h \leftrightarrow p$ , and the system stutters.  $M_{ex2} := \langle \{o, h, p, c\}, c = 0, (c < j \rightarrow c' = c + 1) \wedge (c = j \rightarrow c' = j \wedge o' \leftrightarrow (h \leftrightarrow p)) \rangle$  and the formula  $\varphi_{ex2} := \forall x \exists y. \mathbf{G}((o[x] \leftrightarrow o[y]) \wedge (h[x] \leftrightarrow \neg h[y]) \wedge (p[x] \leftrightarrow \neg p[y]))$  obtained removing the  $ch[x] \leftrightarrow ch[y]$  from  $\varphi_{ex1}$  (see Ex. 2).

This example is not inductive because the property is not strong enough. The counters of the universally quantified self-product can have different values, and if  $c[x] < j$  and  $c[y] = j$ , so the inductive proof obligation fails, as demonstrated by the following counterexample to induction (CTI), where  $j$  is instantiated to 6:  $(c[x] = 3, o[x], h[x], \neg p[x]), (c[x]' = 4, o[x]', h[x]', \neg p[x]'), (c[y] = 6, o[y], \neg h[y], p[y])$ . Intuitively, there is no successor for  $y$  that sets  $o$  to true, because

**Algorithm 2:**  $K$ -induction algorithm**Input:** A STS  $M = \langle V, I, T \rangle$  and a formula  $\varphi := \forall x. \exists y. \mathbf{G}\psi$ 


---

```

1 for  $i=0; true; i++$  do
2   if  $BMC_i^{\forall\exists}$  does not hold then return INVALID
3   if  $\widehat{KIND}_{i+1}^{\forall\exists}$  holds then return VALID

```

---

$h \neq p$ . On the other hand, it is easy to see that this example is  $j$ -inductive, since any path over the self-product of length  $j$  will reach a configuration in which both  $c[x]$  and  $c[y]$  equal  $j$ .

$$\widehat{KIND}_k^{\forall\exists} := \forall V[x]_0^k \forall V[y]_0^{k-1} \exists V[y]_k. \text{Unroll}_k(\overline{V[x]}) \wedge \text{Unroll}_{k-1}(\overline{V[y]}) \wedge \bigwedge_{0 \leq i < k} P_i \wedge \bigwedge_{0 \leq i < j < k} (\text{Diff}(V[x] \cup V[y], i, j)) \rightarrow T(V[y]_{k-1}, V[y]_k) \wedge P_k$$

where  $\text{Diff}(S, i, j) := \neg \bigwedge_{s \in S} (s_i = s_j)$ .  $\widehat{KIND}_k^{\forall\exists}$  extends  $KIND_k^{\forall\exists}$  by adding a constraint that prevents state repetition across the self-composition (over  $V[x]$  and  $V[y]$ ). This constraint, commonly referred to as the simple path condition in classical  $k$ -induction, ensures that the paths considered are loop-free. Enforcing this condition is essential for guaranteeing the completeness of the  $k$ -inductive proof. Consider for instance Ex. 1, for any  $k \in \mathbb{N}$  the proof obligation  $KIND_k^{\forall\exists}$  would fail because any path starting with  $ch[x] \neq ch[y]$  would lead to a CTI because  $y$  is forced to pick a specific value for  $o[x]_k$  if  $ch$  is  $\top$  while  $ch[x]_k$  can be both true or false. In this specific case, the simple path constraint will eventually block all the undesirable configurations.

**Lemma 2.** *If there is a  $k > 0$  such that  $BMC_{k-1}^{\forall\exists}$  and  $\widehat{KIND}_k^{\forall\exists}$  hold, then  $M \models \alpha$*

**Lemma 3.** *If  $M \models \forall x \exists y. \mathbf{G}\psi$ , then there exists a  $k > 0$  such that  $BMC_{k-1}^{\forall\exists}$  and  $\widehat{KIND}_k^{\forall\exists}$  hold.*

The following theorem follows directly from Lemma 1, Lemma 2 and Lemma 3.

**Theorem 2.** *Algorithm 2 algorithm is sound and complete.*

We can easily extend the result for  $\forall x \exists y. \mathbf{G}\psi$  to any  $\forall\exists$  temporal-safety formula.

**Theorem 3.** *Let  $\forall x \exists y. \psi$  be a temporal-safety HyperLTL formula. Then,  $M \models \forall x. \exists y. \psi$  iff*

$$M \times M_{\widehat{\psi}} \models \forall x. \exists y. \mathbf{G} \left( \bigwedge_{v_p \in \text{Pred}(\psi)} (v_p[y] \leftrightarrow p(x, y)) \right)$$

where  $\widehat{\psi}$  is the LTL formula over the symbols  $\text{Pred}(\psi)$  obtained by replacing each top-level predicate  $p$  over  $x$  and  $y$  with a fresh variable  $v_p$ ;  $M_{\widehat{\psi}}$  is the STS obtained through the symbolic automata construction for LTL over finite traces.

### 3.3 Refining Counter-examples To Induction

We define a counter-example to induction (CTI) as a triple of assignments  $(X^{cex}, X^{cex'}, Y^{cex})$  over  $V[x], V[x']$  and  $V[y]$  such that  $P(X^{cex}, Y^{cex})$  and  $T(X^{cex}, X^{cex'})$  hold, and no successor  $Y^{cex'}$  of  $Y^{cex}$  exists such that  $P(X^{cex'}, Y^{cex'})$  holds. Ex. 2 shows an example of a counter-example to induction for property  $\varphi_{ex1'}$ . In this section, our aim is to find strengthenings of the formula derived by counter-examples to induction to either prove or disprove the property.

**Definition 1.** Let  $\psi, \psi'$  be two propositions over  $V[x], V[y]$ . We say that  $\psi'$  is a strengthening of  $\psi$  (over  $M$ ) whenever (1)  $\psi' \rightarrow \psi$ , (2)  $\psi \not\rightarrow \psi'$ , and (3)  $M \models \forall x \exists y. \mathbf{G}\psi$  if and only if  $M \models \forall x \exists y. \mathbf{G}\psi'$ .

Alg. 3 generalizes Alg. 1 using function *genCTI* that generates CTIs.

**Lemma 4.** If *genCTI* returns a strengthening of  $P$  over  $M$ , then Alg. 3 is sound and complete.

A simple way to generalize a counter-example to induction to get a strengthening of  $P$  is to block the assignments  $X^{cex}$  and  $Y^{cex}$  of  $V[x]$  and  $V[y]$ , as shown in Alg. 4. This naive generalization blocks a state over the self-composition of  $M$ .

More efficient techniques aim to block larger portions of the state space of the self-product. Alg. 5 shows a generalization technique based on Craig interpolation and quantifier elimination. It starts from the triple  $(X^{cex}, X^{cex'}, Y^{cex})$  and computes, via interpolation, a formula  $Q(V[x], V[x]', V[y])$  that captures the conflict between the two paths: there is no  $V[y]'$  such that  $\psi(V[x]', V[y]')$  holds. To eliminate  $V[x]'$ , we compute the preimage of  $Q$  using quantifier elimination. As in the naive case, the result is used to strengthen  $P$  in a way that excludes problematic behaviors without violating the original property.

**Theorem 4.** Alg. 4 and Alg. 5 return strengthenings of  $P$  over  $M$ .

*Proof.* (Sketch)

- (naive) The idea is that the tuple  $(X^{cex}, Y^{cex})$  represents a pair of states such that there exists a successor of  $x$  for which no corresponding successor of  $y$  satisfies  $\psi$ . If  $M$  satisfies the original property, then either  $X^{cex}$  is unreachable, or for every successor of  $X^{cex}$ , there exists an alternative assignment  $\bar{Y}^{cex}$  such that the successor of  $y$  satisfies  $\psi$ . This condition ensures that blocking the pair  $(X^{cex}, Y^{cex})$  does not erroneously eliminate a behavior that is required for the property to hold.
- (itp) The interpolation-based approach follows a similar intuition. It starts from the triple  $(X^{cex}, X^{cex'}, Y^{cex})$  and computes, via interpolation, a formula  $Q(V[x], V[x]', V[y])$  that captures the conflict between the two paths: there is no  $V[y]'$  such that  $\psi(V[x]', V[y]')$  holds. To eliminate  $V[x]'$ , we compute the preimage of  $Q$  using quantifier elimination. As in the naive case, the result is used to strengthen  $P$  in a way that excludes problematic behaviors without violating the original property.

□

### 3.4 Incrementality

One of the advantages of classical  $k$ -induction lies in the incrementality of the verification (see e.g. [18]). In this section, we present our incremental algorithm to verify induction and

---

**Algorithm 3:** Induction alg. with generalization function *genCTI*.

---

**Input:** A STS  $M = \langle V, I, T \rangle$  and a formula  $\varphi := \forall x. \exists y. \mathbf{G}\psi$

---

```

1 while true do
2   if  $\not\models \text{BASE}^{\forall\exists}$  then return INVALID
3   else if  $\models \text{IND}^{\forall\exists}$  then return VALID
4   else  $P := \text{genCTI}(P, (X^{cex}, Y^{cex}, X^{cex'}))$ 

```

---

---

**Algorithm 4:** A naive algorithm to strengthen  $P$  using CTI.

---

**Input:** A CTI  $X^{ceex}, X^{ceex'}, Y^{ceex}$ , a STS  $M$  and  $P(V[x], V[y])$ 
**Output:** A strengthening of  $P$  w.r.t.  $M$ 
**1 return**  $P \wedge \neg(X^{ceex} \wedge Y^{ceex})$ 


---

$k$ -induction tailored for SMT solvers that support incremental verification of quantified formulae. To enable incremental reasoning, the proof obligations are first rewritten by removing the outermost universal quantifiers. The remaining existential quantifiers are then pushed inward, introduced only over the sub-formulas that contain their corresponding variables, thus preserving the formula's closure. Finally, negation is applied to transform the validity of the implication into a satisfiability problem over a conjunction. The conjunctive proof obligation for  $k$ -induction (without simple path) is then:

$$Unroll_k(\overline{V[x]}) \wedge Unroll_{k-1}(\overline{V[y]}) \wedge \bigwedge_{0 \leq i < k} P_i \wedge \neg \exists V[y]_k. [T(V[y]_{k-1}, V[y]_k) \wedge P_k]$$

Similarly, the conjunctive proof obligation for BMC is:

$$I(V[x]) \wedge Unroll_k(\overline{V[x]}) \wedge \neg \exists V[y]_0, \dots, V[y]_k. [\bigwedge_{0 \leq i \leq k} P_i \wedge I(V[y]) \wedge Unroll_k(\overline{V[y]})]$$

Regarding BMC, the potential for improvement through incrementality is limited to the unrolling up to  $Unroll_k(\overline{V[x]})$ . The structure of the problem with the increasing internal quantification makes it harder to reason incrementally.

On the other hand, the  $k$ -induction proof obligation has only one inner quantification. By fixing the index in the quantified conjunction and progressively adding constraints to the universally quantified part of the formula, the proof can be incrementally constructed. The approach is detailed in Alg. 6, which enables solving  $k$ -induction (without refinement).

## 4 Implementation and Experimental Evaluation

We implemented the algorithms presented in this paper in a prototype tool, implemented in Python3 and leveraging the PySMT [21] library as the main interface to various SMT solvers. Our tool uses Z3 [16] for the verification of quantified SMT formulas and for performing quantifier elimination. For interpolation we use the default solver backend for interpolation of PySMT (MathSAT5[12]). For QBF solving we use QuAbs [23]. Our tool implements several techniques:

- **Induction with Refinement:** It implements the method described in Section 3.1 with the refinement of Section 3.3.

---

**Algorithm 5:** Generalization using quant. elim. and Craig interpx.

---

**Input:** A CTI  $X^{ceex}, X^{ceex'}, Y^{ceex}$ , a STS  $M$  and  $P(V[x], V[y])$ 
**Output:** A strengthening of  $P$  w.r.t.  $M$ 
**1**  $Q(V[x], V[x'], V[y]) :=$   
 $\quad Itp(X^{ceex} \wedge X^{ceex'} \wedge Y^{ceex}, T(V[x], V[x']) \wedge T(V[y], V[y']) \wedge P(V[x'], V[y']) \wedge P(V[x], V[y]))$   
**2 if**  $getVars(Q) \cap V[x]'$  **then**  
**3**  $\quad Qpre(V[x], V[y]) := QElim(\exists V[x]'. [T(V[x], V[x']) \wedge Q(V[x], V[x'], V[y])])$   
**4 else**  $Qpre := Q$   
**5 return**  $P \wedge \neg Qpre$ 


---

**Algorithm 6:** Incremental  $k$ -induction for hyperproperties

---

**Input:** A STS  $M$ , a formula  $\forall x \exists y. \mathbf{G}\psi$   
**Output:**  $M \models \forall x \exists y. \mathbf{G}\psi$ ?

```

1  $solverKind = IncrSolver(), solverBMC = IncrSolver(), P := \psi, res := UNK$ 
2 for ( $i=0; res = UNK; i++$ ) do
3   if ( $i == 0$ ) then  $solverBMC.addAssert(I(V[x]_0))$ 
4   else  $solverBMC.addAssert(T(V[x]_{i-1}, V[x]_i))$ 
5    $ex_\varphi := \exists V[y]_0, \dots, V[y]_i. Unroll_k(\overline{V[y]}) \wedge \bigwedge_{0 \leq j \leq i} P(V[x]_i, V[y]_i)$ 
6   if  $solverBMC.solve(\neg ex_\varphi) == SAT$  then  $res = INVALID$ 
7   else
8     if ( $i == 0$ ) then
9        $ex_\varphi := \exists V[y]_i. T(V[y]_{i+1}, V[y]_i) \wedge P(V[x]_i, V[y]_i)$   $solverKind.addAssert(\neg ex_\varphi)$ 
10    else  $solverKind.addAssert(T(V[y]_i, V[y]_{i+1}))$ 
11     $solverKind.addAssert(T(V[x]_{i+1}, V[x]_i) \wedge P(V[x]_{i+1}, V[y]_{i+1}))$ 
12    for ( $j = 1; j < i; j++$ ) do
13       $solverKind.addAssert(Diff(V[x] \cup V[y], i, j))$ 
14    if  $\neg solverKind.solve()$  then  $res = VALID$ 
15 return  $res$ 

```

---

- **$k$ -Induction:** It implements the method of Section 3.2 with and without refinement. When refinement is enabled, the tool refines the CTI after each check of  $k$ -induction.
- **Incremental Techniques:** It implements  $k$ -induction without refinement in the incremental fashion described in Section 3.4 (only for Z3).

The input to our tool are system models specified in the SMV input language, extended with HyperLTL formulas. This syntax is closely aligned with the one used by the tools **HyperQB** [25] and **AutoHyper** [6]. Our tool supports both finite and infinite state systems, as long as they can be expressed using theories supported by Z3 (e.g., linear integer and linear real arithmetic).

## 4.1 Empirical Evaluation

We evaluate our prototype on a diverse benchmark suite, collected from **AutoHyper** [6], **HyperQB** [25], **HyPro** [4] and **HyperATL\*** [3, 7]. We also consider a smaller benchmark suite of infinite-state benchmarks from **HyHorn** [27]. The benchmarks<sup>1</sup> cover a variety of property types and systems including:

- **Path planning and robotic robustness** [25, 6]: Given a grid with a set of obstacles, a starting position and a goal position, the shortest path property is  $\exists x \forall y. \neg goal[y] \mathbf{U} goal[x]$ . The second property considers a set of starting positions and a set of goal positions, and states whether there is a single plan to reach a goal from all the starting positions. Both are  $\forall \exists$  safety properties checked over grids of various sizes.
- **Symmetry in mutual exclusion protocols** [25, 6]: variants of *Lamport's bakery algorithm* enriched with symmetric treatment of processes. The original Bakery implementation violates symmetry because if the process with lower pid is favoured. A modified model uses a flip variable and, if the flip variable is true, we compare the PID in increasing

<sup>1</sup>The results of the experimental evaluation can be found at <https://drive.proton.me/urls/H2M2H2K38C#Cm5swFdqJERe>.

order. We consider models with 2, 3 and 4 processes with a property that also encodes strategy in the formula (i.e.  $flip[x] \leftrightarrow \neg flip[y]$ ) and a version without it.

- **Non-interference in Boolean programs** [4, 3, 7]: *Generalized non-interference (GNI)* over non-terminating Boolean programs with varying bit-widths (e.g., 2, 4, and 6 bits). The models are expressed as Boolean programs (only supported by AutoHyper) and SMV.
- **Infinite-state non-interference and Refinement properties** [27]: Variants of non-interference properties over integers, compiler optimizations properties and programs with non-deterministic choices. These models were designed to be asynchronous, adapted to be amenable for our tool (most of the models required no changes).

Since our tool supports only  $\forall^*\exists^*.\varphi_S$ , the benchmarks are restricted to this fragment. The evaluation includes a comparison with **AutoHyper** and **HyPro**, a comparison with **HyperQB**, a comparison with **HyHorn** and an analysis of the prototype’s configurations. We extended the benchmarks increasing the size of the programs and increasing (and reducing) the state space to various configurations to better analyze the performance of each approach. Experiments for our tool, **AutoHyper** and **HyPro** were conducted on a SLURM-managed high-performance computing cluster. Each job was allocated a single node with homogeneous hardware: Intel Xeon Gold 6226R CPUs running at 2.9GHz, with a strict runtime limit of 2 hours per job, a limit to single core usage and a memory cap of 8GB. Due to cluster policy and dependency isolation requirements, both tools were executed within **Singularity** containers. Beyond tool-to-tool comparison, we conducted an in-depth evaluation of the various verification strategies supported by our prototype, including plain induction,  $k$ -induction, and their respective combinations with refinement and incremental solving, as outlined in Section 3.3. This analysis aims to assess the impact of refinement techniques and incremental solving across a pool of benchmarks.

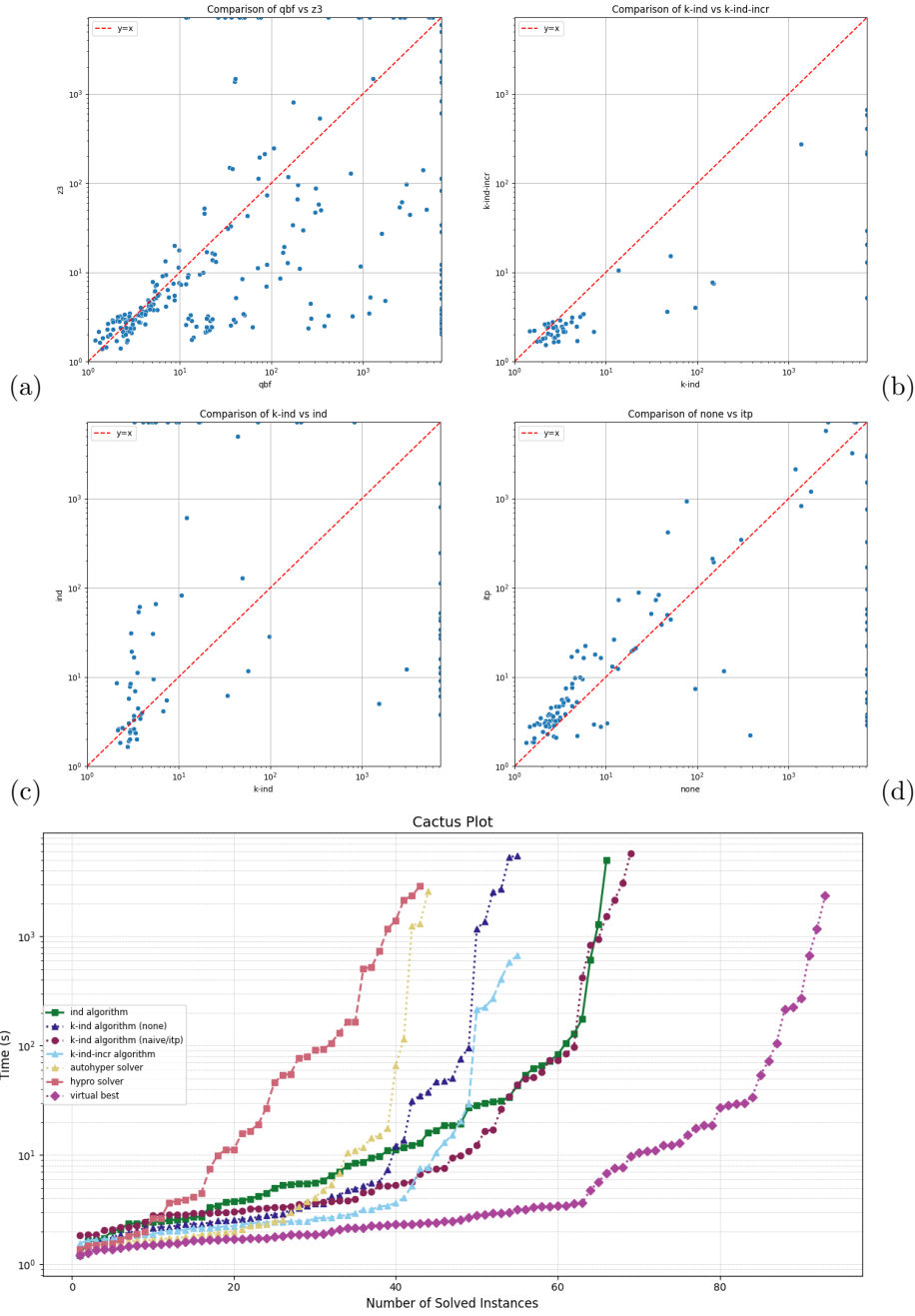
In contrast, experiments involving **HyperQB** [25] and **HyHorn** were conducted on a dedicated Linux machine (Intel Core i5-1135G7 @ 2.40,GHz, 32,GB RAM) outside the cluster environment, with an 8,GB memory limit and a 5-minute timeout. This decision was made due to technical issues in setting up dependencies of these tools on the cluster. As **HyperQB** supports only falsification of HyperLTL formulas, we restricted this comparison to a subset of benchmarks containing known counterexamples. For the comparison with **HyHorn**, we considered an adaptation of the benchmark suite used in its experimental evaluation [27], which consists of infinite-state benchmarks under asynchronous semantics. Since our approach assumes synchronous semantics, we restricted the comparison to benchmarks admitting an equivalent synchronous encoding. Most benchmarks could be used unchanged; for `compiler_opt_2` and `async_gni`, we introduced stuttering transitions to align traces and preserve the intended behavior under synchronous semantics, while two additional benchmarks were excluded because no suitable synchronous encoding could be obtained. Furthermore, some benchmarks in **HyHorn** require user-provided under-approximations (called *restrictions*) to bound nondeterministic choices, whereas induction (**HyperInd**) and  $k$ -induction (**HyperKind**) can be applied directly to the corresponding unrestricted models.

**Comparison with HyPro and AutoHyper.** The overall performance results are summarized in the cactus plot in Fig. 1(e), with detailed tables available in Appendix B. The maximum number of solved instances are obtained by  $k$ -induction with refinement and induction with refinement. However, the various configurations and tools exhibit complementary results, as evidenced by the virtual best solver significantly outperforming any individual approach. Both  $k$ -induction and induction consistently outperformed AutoHyper on a substantial subset of benchmarks. We attribute this to the scalability limitations of explicit-state techniques, which often struggle with larger systems. In contrast, our symbolic approach, especially when

combined with refinement, is able to generalize properties more effectively and provide invariants on larger benchmarks. For example, while AutoHyper performs well on non-interference benchmarks involving Boolean programs with few bits, its performance degrades significantly as the bit-width increases (e.g., 4–6 bits), often resulting in timeouts or out-of-memory failures. On the other hand, AutoHyper outperforms our tool on some benchmarks, particularly those involving robust path planning and shortest path properties. These examples require deep unrolling to uncover counterexamples—an area where our BMC-based method is less efficient. Conversely, when properties are false and the counterexamples are short, our approach is often significantly faster than AutoHyper. Our experimental evaluation also highlights notable differences in performance between our approach and HyPro. While HyPro shares some of the scalability limitations observed in AutoHyper—solving fewer benchmarks overall—HyPro stands out in some specific cases. Notably, HyPro was the only tool that handles the Bakery benchmarks with three processes without requiring a user-provided strategy, which suggests that prophecy-based strategy synthesis (implemented in HyPro) can serve as a powerful refinement technique in certain scenarios. On the other hand, HyPro struggled significantly on larger or more complex instances and was unable to handle many false benchmarks.

**Comparison of various configurations.** Fig. 1(a) shows how the solvers exhibit complementary strengths: QBF is more effective on path-planning, even without incrementality, while Z3 performs better on software-related benchmarks. Due to technical limitations, we were unable to use QuAbs for incremental property checking and therefore focused on induction and  $k$ -induction proof obligations. As Fig. 1(b) shows, the  $k$ -ind-incr variant consistently outperforms plain  $k$ -ind, demonstrating the effectiveness of incremental solving. Even in BMC settings, where incrementality is less naturally applicable, we observed noticeable speedups. These results suggest that extending incremental solving to induction with refinement and  $k$ -induction with refinement could be a promising direction for future work. From Fig. 1(c), we observe that  $k$ -induction with refinement and induction with refinement produce complementary results. Specifically,  $k$ -induction performs better on invalid benchmarks, where unrolling helps eliminate large portions of counterexamples. However, unrolling increases the complexity of proof obligations, especially without incremental solving, and can be ineffective in complex models such as Bakery, where it fails to eliminate enough bad configurations. In such cases, induction with interpolation-based refinement can be more effective in filtering undesired behaviors. Similarly, in GNI benchmarks, where key variables are not explicitly referenced in the property,  $k$ -induction struggles, whereas standard induction with refinement can still succeed. We also analyze the role of refinement in  $k$ -induction. Fig. 1(d) compares  $k$ -induction with and without interpolation-based refinement. Although refinement introduces some overhead by increasing property complexity, it consistently improves performance and often outperforms the non-refined version.

**Comparison with HyperQB.** Table 1 shows the comparison between HyperKind and HyperQB. We observe that, on the larger robotic robustness benchmarks, HyperQB outperforms our tool when using Z3 as backend, while in our implementation HyperKind with Z3 does not terminate within the timeout unless incremental verification is enabled. A possible explanation is that HyperQB employs a more specialized handling of the transition relation, whereas HyperKind currently relies on a translation through the VMT representation. Supporting this hypothesis, we observed that an earlier version of HyperQB was significantly slower than the current implementation, despite relying on the same overall verification methodology. We attribute this to recent optimizations introduced in HyperQB to better handle input formats. Overall,



(e) Comparing the various configurations over all the instances. The x-axis represents solved instances, and the y-axis shows solving time. Virtual best considers the minimum time taken by each configuration/tool.

Figure 1: Results of the empirical evaluation

incrementality already closes a substantial portion of the gap with **HyperQB**, and we can still prove properties with limited overhead, especially under incremental verification.

**Comparison with HyHorn.** Table 2 and Table 3 shows the results of the HyHorn benchmarks. HyHorn substantially outperforms our approach on `compiler_opt_2` and `async_gni`, suggesting that CHC-based reasoning is particularly effective at discovering suitable relations between traces. On many of the remaining benchmarks, however, HyperInd and HyperKind achieve comparable runtimes, and in several cases outperform both HyHorn configurations. Interestingly, predicate abstraction appears to have a substantial impact only on a small subset of benchmarks, notably `counter_diff` and `paper_fig3`, while providing limited benefit on most others. We also observed that the restrictions required by HyHorn often improve the performance of our approach as well (we include them in the property); for instance, they turn timeouts into solved instances on benchmarks such as `p2_gni` and `paper_fig3`. Unlike HyHorn, however, HyperInd and HyperKind can be applied directly to unrestricted models and therefore do not depend on user-provided under-approximations. Taken together, these results suggest that induction-based reasoning provides a competitive alternative to CHC-based verification for symbolic hyperproperties, while the two approaches exhibit complementary strengths.

## 5 Conclusion

We introduced in this paper a novel approach for the verification of  $\forall\exists$  temporal-safety properties. By extending classical induction and  $k$ -induction techniques to operate over sets of traces, we developed generalized proof rules tailored to the hyperproperty setting. Our method introduces the notion of hyper- $(k)$ -inductive invariants as certificates of property satisfaction and provides refinement strategies to strengthen specifications into inductive form. We implemented a prototype tool and evaluated it against state-of-the-art solvers on a range of benchmarks. The empirical evaluation suggests that our technique is both practical and complementary to exist-

| benchmark/tool   | hyperQB_qbf  | hyperQB_z3   | hyperkind          | hyperkind_incr      | hyperkind_qbf |
|------------------|--------------|--------------|--------------------|---------------------|---------------|
| mutation         | 0.16         | 0.18         | 0.03               | 0.02                | <b>0.01</b>   |
| robo_robust_100  | 5.24         | 4.89         | TO                 | <b>2.66/3.78</b>    | 7.12/13.01    |
| robo_robust_1600 | 50.170       | <b>16.80</b> | TO                 | <b>16.73/</b> 21.52 | 35.49/64.30   |
| robo_robust_3600 | 82.92        | <b>29.77</b> | TO                 | <b>26.73/</b> 37.26 | 54.76/107.25  |
| snark            | <b>7.113</b> | 8.715        | <b>5.48/</b> 11.85 | 5.89/9.47           | 12.81/26.73   |

Table 1: Comparison of cumulative verification times between hyperkind and **HyperQB**. For  $k$ -induction, entries of the form  $x/y$  denote the BMC time ( $x$ ) and the total time.

| benchmark/tool | hyhorn | hyhorn-no-pa | hyperind    | hyperkind   |
|----------------|--------|--------------|-------------|-------------|
| async_gni      | 1.14   | <b>1.09</b>  | 22.23       | 5.85        |
| non_det_add    | 2.35   | 1.76         | <b>0.96</b> | 1.06        |
| p1_gni_with_C  | TO     | TO           | TO          | TO          |
| smaller        | 0.68   | 1.87         | 1.16        | <b>0.65</b> |

Table 2: Result of the evaluation over HyHorn’s models with bounded non-deterministic choices.

| benchmark/tool | hyhorn      | hyhorn-no-pa | hyperind | hyperind_r | hyperkind   | hyperkind_r |
|----------------|-------------|--------------|----------|------------|-------------|-------------|
| compiler_opt_1 | 0.77        | 0.77         | 0.79     | -          | <b>0.61</b> | -           |
| compiler_opt_2 | <b>0.94</b> | 1.74         | TO       | -          | TO          | -           |
| counter_diff   | 0.56        | 16.21        | 1.72     | 0.66       | 0.39        | <b>0.33</b> |
| counter_negate | 0.49        | 0.50         | 0.45     | 0.44       | 0.72        | <b>0.31</b> |
| counter_sum    | 0.58        | 0.86         | 0.67     | 1.61       | <b>0.33</b> | 0.62        |
| p1_gni         | 1.31        | 1.65         | 22.48    | -          | <b>1.22</b> | -           |
| p1_simple      | 0.89        | 1.49         | 4.43     | -          | <b>0.76</b> | -           |
| p2_gni         | 10.33       | 8.70         | 59.52    | 23.68      | TO          | <b>3.81</b> |
| p3_gni         | 1.24        | 1.12         | 1.57     | -          | <b>0.89</b> | -           |
| p4_gni         | 2.23        | <b>1.96</b>  | 12.81    | -          | 5.69        | -           |
| paper_fig3     | 2.02        | 156.38       | TO       | 5.65       | TO          | <b>0.82</b> |
| refine         | 0.63        | 0.60         | 0.57     | 0.63       | <b>0.43</b> | 0.46        |
| refine_2       | <b>0.66</b> | 0.96         | 1.10     | -          | 0.73        | -           |

Table 3: Results of the evaluation over HyHorn’s models with non-deterministic choice of integers. Column *hyperind\_r* and *hyperkind\_r* represent respectively instances of induction and  $k$ -induction with modified properties to include hyhorn restrictions. - denotes benchmarks for which restrictions are already included in the property.

ing tools.

There are several promising directions for future work. First, we aim to explore how abstraction techniques can be integrated into our framework, following approaches similar to those in [34] and [5]. We are also interested in adapting other existing techniques, such as Interpolation-Based Model Checking [30] and IC3 [10], to HyperLTL model-checking. A particular challenge with IC3 is that, while it typically over-approximates the state space, verifying  $\forall\exists$ -hyperproperties may require both over- and under-approximating at the same time. Finally, we plan to improve incrementality by integrating incremental QBF-solving techniques [29], enabling the dynamic addition of new predicates in closed QBF formulas. We also plan to explore bounded Skolem functions generated by QBF solvers during bounded model checking to further accelerate verification.

## References

- [1] G. Barthe, P. R. D’Argenio, and T. Rezk. Secure information flow by self-composition. *Mathematical Structures in Computer Science*, 21:1207–1252, 2011.
- [2] J. Baumeister, N. Coenen, B. Bonakdarpour, B. Finkbeiner, and C. Sánchez. A temporal logic for asynchronous hyperproperties. In *Proc. of the 33rd Int’l Conf. on Computer Aided Verification (CAV’21), Part I*, volume 12759 of *LNCS*, pages 694–717. Springer, 2021.
- [3] R. Beutner and B. Finkbeiner. A Temporal Logic for Strategic Hyperproperties. In *Proc. of the 32nd Int’l Conf. on Concurrency Theory (CONCUR 2021)*, volume 203 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 24:1–24:19, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [4] R. Beutner and B. Finkbeiner. Prophecy variables for hyperproperty verification. In *Proc. of the 2022 IEEE 35th Computer Security Foundations Symposium (CSF’22)*, pages 471–485, 2022.
- [5] R. Beutner and B. Finkbeiner. Software verification of hyperproperties beyond  $k$ -safety. In *Proc. of the 34th Int’l Conf. on Computer Aided Verification (CAV’22), Part I*, volume 13371 of *LNCS*,

- pages 341—362. Springer, 2022.
- [6] R. Beutner and B. Finkbeiner. AutoHyper: Explicit-state model checking for hyperltl. In *Proc. of the 29th Int'l Conf. on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'23), Part I*, volume 13993 of *LNCS*, pages 145–163. Springer, 2023.
  - [7] R. Beutner and B. Finkbeiner. HyperATL\*: A logic for hyperproperties in multi-agent systems. *Logical Methods in Computer Science*, 19, May 2023.
  - [8] A. Biere, A. Cimatti, E. M. Clarke, and Y. Zhu. Symbolic model checking without BDDs. In *Proc. of the 5th Int'l Conf. on Tools and Algorithms for Construction and Analysis of Systems (TACAS'99)*, volume 1579 of *LNCS*, pages 193–207. Springer, 1999.
  - [9] B. Bonakdarpour, C. Sánchez, and G. Schneider. Monitoring hyperproperties by combining static analysis and runtime verification. In *Proc. of the 8th Symp. on Leveraging Applications of Formal Methods, Verification and Validation (ISoLA'18)*, volume 11245 of *LNCS*, pages 8–27. Springer-Cham, 2018.
  - [10] A. R. Bradley. SAT-based model checking without unrolling. In *Proc. of the 12th Int'l Conf. on Verification, Model Checking, and Abstract Interpretation (VMCAI'11)*, volume 6538, pages 70–87. Springer, 2011.
  - [11] M. Brain, S. Joshi, D. Kroening, and P. Schrammel. Safety verification and refutation by  $k$ -invariants and  $k$ -induction. In *Proc. of the 22nd Int'l Symp. on Static Analysis (SAS'15)*, volume 9291 of *LNCS*, pages 145–161. Springer, 2015.
  - [12] A. Cimatti, A. Griggio, B. Schaafsma, and R. Sebastiani. The MathSAT5 SMT solver. In *Proc. of the 19th Int'l Conf. Tools and Algorithms for the Construction and Analysis of Systems (TACAS'13)*, volume 7795 of *LNCS*, pages 93–107. Springer, 2013.
  - [13] M. R. Clarkson, B. Finkbeiner, M. Koleini, K. K. Micinski, M. N. Rabe, and C. Sánchez. Temporal logics for hyperproperties. In *Proc. of the 3rd Conference on Principles of Security and Trust (POST 2014)*, volume 8414 of *LNCS*, pages 265–284. Springer, 2014.
  - [14] M. R. Clarkson and F. B. Schneider. Hyperproperties. *Journal of Computer Security*, 18(6):1157–1210, 2010.
  - [15] N. Coenen, B. Finkbeiner, C. Sánchez, and L. Tentrup. Verifying hyperliveness. In *Proc. of the 31st Int'l Conf. on Computer Aided Verification (CAV'19)*, volume 11561 of *LNCS*, pages 121–139. Springer, 2019.
  - [16] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Proc. of the 14th Int'l Conf. on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'08)*, volume 4963 of *LNCS*, pages 337–340. Springer, 2008.
  - [17] A. Duret-Lutz, E. Renault, M. Colange, F. Renkin, A. Gbaguidi Aisse, P. Schlehuber-Caissier, T. Medioni, A. Martin, J. Dubois, C. Gillard, and H. Lauko. From Spot 2.0 to Spot 2.10: What's new? In *Proc. of the 34th Int'l Conf. on Computer Aided Verification (CAV'22). Part II*, volume 13372 of *LNCS*, pages 174–187. Springer, 2022.
  - [18] N. Eén and N. Sörensson. Temporal induction by incremental SAT solving. In *Proc. of the First Int'l Workshop on Bounded Model Checking (BMC@CAV'03)*, volume 89(4) of *Electronic Notes in Theoretical Computer Science*. Elsevier, 2003.
  - [19] C. Eisner, D. Fisman, J. Havlicek, Y. Lustig, A. McIsaac, and D. V. Campenhout. Reasoning with temporal logic on truncated paths. In *Proc. of the 15th Int'l Conf. on Computer Aided Verification (CAV'03)*, volume 2725 of *LNCS*, pages 27–39. Springer, 2003.
  - [20] B. Finkbeiner, M. Rabe, and C. Sánchez. Algorithms for model checking HyperLTL and HyperCTL\*. In *Proc. of the 27th Int'l Conf. on Computer Aided Verification (CAV'15)*, volume 9206 of *LNCS*, pages 30–48. Springer, 2015.
  - [21] M. Gario and A. Micheli. PySMT: a solver-agnostic library for fast prototyping of SMT-based algorithms. In *SMT Workshop 2015*, 2015.
  - [22] J. A. Goguen and J. Meseguer. Security policies and security models. In *Proc. of the IEEE Symposium on Security and Privacy (S&P'82)*, pages 11–20. IEEE, 1982.

- [23] J. Hecking-Harbusch and L. Tentrup. Solving QBF by abstraction. In *Proc. of the Ninth Int'l Symp. on Games, Automata, Logics, and Formal Verification (GandALF'18)*, volume 277 of *EPTCS*, pages 88–102, 2018.
- [24] T.-H. Hsu, B. Bonakdarpour, B. Finkbeiner, and C. Sánchez. Bounded model checking for asynchronous hyperproperties. In *Proc. of the 29th Int'l Conf. on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'23), Part I*, volume 13993 of *LNCS*, pages 29–46. Springer, 2023.
- [25] T.-H. Hsu, C. Sánchez, and B. Bonakdarpour. Bounded model checking for hyperproperties. In *Proc. of the 27th Int'l Conf on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'21). Part I*, volume 12651 of *LNCS*, pages 94–112. Springer, 2021.
- [26] T.-H. Hsu, S. Sheinvald, C. Sánchez, and B. Bonakdarpour. Efficient loop conditions for bounded model checking hyperproperties. In *Proc. of the 29th Int'l Conf. on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'23)*, volume 13993 of *LNCS*, pages 66–84. Springer, 2023.
- [27] S. Itzhaky, S. Shoham, and Y. Vizel. Hyperproperty verification as CHC satisfiability. In *Proc. of the 33rd European Symp. on Programming Languages and Systems (ESOP'24), Part II*, volume 14577 of *LNCS*, pages 212–241. Springer, 2024.
- [28] T. Kahsai and C. Tinelli. PKind: A parallel k-induction based model checker. In *Proc. of the 10th Int'l Workshop on Parallel and Distributed Methods in verifiCation (PDMC'11)*, volume 72 of *EPTCS*, pages 55–62, 2011.
- [29] F. Lonsing and U. Egly. Incremental QBF solving. In *Proc. of the 20th Int'l Conf. on Principles and Practice of Constraint Programming (CP'14)*, volume 8656 of *LNCS*, pages 514–530. Springer, 2014.
- [30] K. L. McMillan. Interpolation and SAT-based model checking. In *Proc. of 15th Int'l Conf. on Computer Aided Verification (CAV'03)*, volume 2725 of *LNCS*, pages 1–13. Springer, 2003.
- [31] L. Pike and S. D. Johnson. The formal verification of a reintegration protocol. In *Proc. of the 5th ACM International Conference On Embedded Software (EMSOFT'05)*, pages 286–289. ACM, 2005.
- [32] A. Pnueli. The temporal logic of programs. In *Proc. of the 18th IEEE Symp. on Foundations of Computer Science (FOCS'77)*, pages 46–67. IEEE CS Press, 1977.
- [33] M. Sheeran, S. Singh, and G. Stålmarck. Checking safety properties using induction and a SAT-solver. In *Proc. of the 3rd Int'l Conf. on Formal Methods in Computer-Aided Design (FMCAD'00)*, volume 1954 of *LNCS*, pages 108–125. Springer, 2000.
- [34] S. Tonetta. Abstract model checking without computing the abstraction. In *Proc. of the Second World Congress on Formal Methods (FM'09)*, volume 5850 of *LNCS*, pages 89–105. Springer, 2009.
- [35] Y. Wang, M. Zarei, B. Bonakdarpour, and M. Pajic. Statistical verification of hyperproperties for cyber-physical systems. *ACM Transactions on Embedded Computing systems (TECS)*, 18(5s):92:1–92:23, 2019.
- [36] S. Zdancewic and A. C. Myers. Observational determinism for concurrent program security. In *Proc. of the 16th IEEE Computer Security Foundations Workshop (CSFW'03)*, pages 29–43. IEEE, 2003.

## A Proofs

### A.1 Proofs of Induction algorithm

**Lemma 5.** *Let  $P := \psi$ , if  $\not\models \text{BASE}^{\forall\exists}$  then  $M \not\models \forall x\exists y.\mathbf{G}\psi$*

#### Proof of Lemma 5

*Proof.* By assumption  $\not\models BASE^{\forall\exists}$ , which means that there is an initial state  $s$  of  $M$  such that for any initial state  $s'$  of  $M$  s.t. together they violate  $P$ . Since  $M$  has no deadlock there exists an infinite trace  $\sigma$  that extends  $s$  ( $\sigma(0) = s$ ). Suppose by contradiction that  $M \models \forall x\exists y.\mathbf{G}\psi$ . Then there exists infinite trace  $\sigma'$  such that the trace assignment  $\Pi$  that assigns  $\sigma$  to  $x$  and  $\sigma'$  to  $y$ , is such that  $\Pi \models \mathbf{G}\psi$ , and consequently the initial states  $\sigma(0)$  and  $\sigma'(0)$  satisfy  $P$ , which is a contradiction. Thus,  $M \not\models \forall x\exists y.\mathbf{G}\psi$ .  $\square$

**Lemma 6.** *if  $\models BASE^{\forall\exists}$  and  $\models IND^{\forall\exists}$  then  $M \models \forall x\exists y.\mathbf{G}\psi$*

### Proof of Lemma 6

*Proof.* We prove the lemma constructively by providing a trace  $\sigma'$  for each trace  $\sigma$ , defined as follows:

- ( $i = 0$ ): Choose  $\sigma'(0)$  to be any initial state such that the pair  $\sigma(0), \sigma'(0)$  satisfies  $\psi$ . The existence of such a state is guaranteed by the assumption  $\models BASE^{\forall\exists}$ .
- Inductive case ( $i > 0$ ): Suppose we have already constructed  $\sigma'(i-1)$  such that  $\sigma(i-1), \sigma'(i-1)$  satisfies  $P$ . Since  $\sigma(i)$  is a successor of  $\sigma(i-1)$  (which exists because  $M$  is deadlock-free), and  $IND^{\forall\exists}$  holds, there exists a successor  $\sigma'(i)$  of  $\sigma'(i-1)$  such that the pair  $\sigma(i), \sigma'(i)$  satisfies  $P$ , and hence also satisfies  $\psi$ .

$\square$

### Proof of Theorem 1

*Proof.* Directly follows from Lemma 5 and Lemma 6.  $\square$

## A.2 Proofs of K-Induction algorithm

**Proof of Lemma 1** There is a  $k \geq 0$  such that  $BMC_k^{\forall\exists}$  does not hold iff  $M \not\models \forall x\exists y.\mathbf{G}\psi$

*Proof.* ( $\Rightarrow$ ) The proof is the same as for Lemma 5 but with a longer path. ( $\Leftarrow$ ) Suppose that  $M \not\models \forall x\exists y.\mathbf{G}\psi$ . Then there is a trace  $x$  and  $j \in \mathbb{N}$  such for any trace  $x'$  of  $M$   $\Pi \not\models \psi$  where  $\Pi$  is the trace assignment assigning  $x$  to the pointed trace  $(x, j)$  and  $y$  to  $(x', j)$ . By assumption  $BMC_j^{\forall\exists}$  holds, which means that for each path of  $M$  of length  $j$  there is a path of length  $j$  such that the product of the two paths always satisfies  $\psi$ . Then, if we consider the finite path build from  $x$  stopping at point  $j$  there must exists a finite path such that the composition of the two paths satisfies  $\psi$ . However, since by assumption  $M$  is deadlock-free, then the “existential” path is extendible to an infinite trace, which contradicts  $M \not\models \forall x\exists y.\mathbf{G}\psi$ .  $\square$

### Proof of Lemma 2

*Proof.* Since  $BMC_{k-1}^{\forall\exists}$  holds, than for each path  $V[x]_0, \dots, V[x]_{k-1}$  there is a path  $V[y]_0, \dots, V[y]_{k-1}$  such that  $P(V[x]_i, V[y]_i)$  holds for each  $i$ . Consider all paths over  $V[x], V[y]$  such that  $P(V[x]_i, V[y]_i)$  holds for each  $i$ . We can partition the set of paths into  $G_1$ , which contains cycles, and  $G_2$  which does not contain cycles. Since these paths are induced by  $BMC_k^{\forall\exists}$  the  $V[x]$  part of  $G_1 \cup G_2$  is exactly the set of all paths of length  $k$ . Since  $\widehat{KIND}_k^{\forall\exists}$  holds, each path of  $G_2$  extended with any  $V[x]_k$  has a  $V[y]_k$  such that  $P(V[x]_k, V[y]_k)$  still holds.

Let  $V[x]_0, V[x]_{k-1}, V[y]_0, \dots, V[y]_{k-1} \in G_1$ . Since there is at least a cycle, we can compress the path obtaining:  $\widehat{V[x]_0, \dots, V[x]_l}, \widehat{V[y]_0, \dots, V[y]_l}$  with  $l < k-1$ . Then if we pick any

$V[x]_k = \widehat{V[x]}_{l+1}$  s.t.  $T(\widehat{V[x]}_l, \widehat{V[x]}_{l+1})$  by  $\models BMC_k^{\forall\exists}$  there is an assignment  $V[y]_0, \dots, V[y]_{l+1}$  for the compressed path satisfying always  $P$  from which we deduce that the response exists for the original path as well.  $\square$

### Proof of Lemma 3

*Proof.* We proceed by contradiction. Suppose that for all  $k > 0$  either  $\not\models BMC_{k-1}^{\forall\exists}$  or  $\not\models \widehat{KIND}_k^{\forall\exists}$ . Since  $M$  satisfies the property, then for all  $k' : BMC_{k'}^{\forall\exists}$ . Therefore,  $\widehat{KIND}_k^{\forall\exists}$  must be violated at each  $k$ . Let  $d$  be the diameter of  $M \times M$  (i.e. the amount of states of  $M \times M$ ). It follows that there are not loop-free paths longer than  $d$  in  $M \times M$ . Then there are no loop free paths over  $V[x], V[y]$  of length  $d + 1$  contradicting that  $\widehat{KIND}_{d+1}^{\forall\exists}$  does not hold.  $\square$

### Sketch Proof of Theorem 3

*Proof.* (Sketch) Intuitively the big and relates the predicates of  $\psi$  to the existential variables of  $M_{\widehat{\psi}}$ . Since  $\widehat{\psi}$  is in safetyLTL the automata construction does not involve fairness constraints. Therefore, bounding the predicates of  $\psi$  with the variables of  $\widehat{\psi}$  is sufficient to have an equivalent model checking problem.  $\square$

## A.3 Proof of Theorem 4

### Proof of Lemma 4

*Proof.* The soundness follows from Thm. 1 and by the fact that *genCTI* returns the strengthening of  $P$  (and indirectly of  $\psi$ ). Completeness follows from the fact that  $M$  is finite state. Suppose that the algorithm does not terminate, then at some point  $P$  will block all the states of the self-product of  $M$ . Therefore,  $BASE^{\forall\exists}$  and  $IND^{\forall\exists}$  will become vacuously true contradicting the assumption that the algorithm does not terminate.  $\square$

**Definition 2.** (*Craig interpolant*) Let  $A$  and  $B$  be two propositions such that  $A \wedge B$  is UNSAT.

A *Craig interpolant*  $I$  is a proposition over the common symbols of  $A$  and  $B$  such that:  $A \Rightarrow I$  and  $I \wedge B$  is UNSAT.

naive: The idea is that the tuple  $(X^{cex}, Y^{cex})$  represents a pair of states such that there exists a successor of  $x$  for which no corresponding successor of  $y$  satisfies  $\psi$ . If  $M$  satisfies the original property, then either  $X^{cex}$  is unreachable, or for every successor of  $X^{cex}$ , there exists an alternative assignment  $\overline{Y^{cex}}$  such that the successor of  $y$  satisfies  $\psi$ . It is easy to see by contradiction: There is a trace  $\sigma$ ,  $i \in \mathbb{N}$  such that for each trace  $\sigma'$   $\sigma(i)$  assigns the same values as  $X^{cex}$  and  $\sigma'(i)$  assigns the same value as  $Y^{cex}$ . The trace assignment  $\Pi_i$  (obtained by mapping  $x$  to  $(\sigma, i)$  and  $y$  to  $(\sigma', i)$ ) satisfies  $\psi$  and, since the original property is satisfied at index  $i + 1$ ; it contradicts the fact that there is no successor for  $Y^{cex}$  satisfying with all successor of  $X^{cex}$   $\psi$ . Since either  $X^{cex}$  is unreachable or there is an alternative  $Y^{cex}$  for each  $X^{cex}$ , then any counter-example for  $\psi \wedge \neg(X^{cex} \wedge Y^{cex})$  is also a counterexample for  $\psi$ .  $\square$

itp: For this case we need to show that  $Qpre$  has the same characteristic of  $X^{cex} \wedge Y^{cex}$  i.e. there is a  $x$  successor of  $Qpre$  such that no successor of  $y$  can satisfy  $\psi$ .

First of all, let  $A = X^{cex} \wedge X^{cex'} \wedge Y^{cex}$  and  $B = T(V[y], V[y']) \wedge P(V[x]', V[y]')$ :  $A \wedge B$  is UNSAT (note that in the original algorithm we add more predicates to heuristically

shrink the interpolant). This occurs because  $(X^{cex}, X^{cex'}, Y^{cex})$  is a CTI and thus there is no  $V[y]'$  such that  $T(V[y], V[y]')$  and  $P(V[x]', V[y]')$ .

Then, following the algorithm, we obtain  $Q(V[x], V[x]', V[y])$  which generalizes the CTI and for which we are guaranteed that there is no  $V[y]'$  satisfying  $T(V[y], V[y]') \wedge P(V[x]', V[y]')$ . If  $Q$  does not contain  $V[x]'$  variables, then  $Qpre$  is  $Q$  and satisfies the desired constraint (there is a successor of  $x$  such that any successor of  $y$  violates  $x$ ). If not, then the pre-image (obtained via quantifier elimination has that characteristic.  $\square$

## B Table Results

In the following, we provide the tables with complete results. Timeout – denoted with TO – is 2 hours while memory out – denoted by MO – is 8GB.

| Instance                     | vb-induction | vb_kind-no-ref | vb-kind-ref | k-ind-incr | AutoHyper | HyPro   |
|------------------------------|--------------|----------------|-------------|------------|-----------|---------|
| 10_NI.big_tini               | TO           | 2710.73        | TO          | 223.92     | MO        | MO      |
| 10_NI.big_tsni               | TO           | 2549.72        | 5771.73     | 213.98     | MO        | MO      |
| 10_NI.huge_tini              | TO           | 5265.11        | TO          | 666.0      | TO        | MO      |
| 10_NI.small_tini             | TO           | 5.54           | 9.47        | 3.38       | 10.55     | 522.08  |
| 10_NI.small_tsni             | TO           | 5.24           | 9.95        | 3.16       | 11.08     | 507.78  |
| 10_NI.tini                   | TO           | 34.79          | 73.84       | 7.56       | MO        | MO      |
| 10_NI.tsni                   | TO           | 37.92          | 84.2        | 7.68       | MO        | MO      |
| bakery_3procs_lang_inclusion | TO           | TO             | TO          | TO         | MO        | 1173.53 |
| bakery_NON_sym_3_procs       | 8.55         | 1.86           | 2.1         | 2.35       | 1313.83   | UNK     |
| bakery_sym_2_procs           | 1292.24      | TO             | TO          | 409.38     | MO        | 53.71   |
| bakery_sym_2_procs_strategy  | 3.79         | 3.58           | 3.85        | 2.15       | MO        | 54.97   |
| bakery_sym_3_procs           | TO           | TO             | TO          | TO         | MO        | 2351.46 |
| bakery_sym_3_procs_strategy  | 128.28       | 47.05          | 49.74       | 3.63       | TO        | 2895.95 |
| buffer_sched                 | TO           | 3.45           | 5.19        | 2.15       | MO        | 2153.78 |
| buffer_sched_big             | TO           | 3.61           | 7.46        | 2.66       | MO        | MO      |
| buffer_sched_huge            | TO           | 4.19           | 16.97       | 2.79       | TO        | MO      |
| buffer_sched_small           | TO           | 2.85           | 4.64        | 2.41       | 66.05     | 45.89   |
| buffer_sched_tiny            | TO           | 2.55           | 3.74        | 1.98       | 1.7       | 7.42    |
| buffer_unsched               | TO           | 3.27           | 5.58        | 2.47       | MO        | TO      |
| buffer_unsched_big           | TO           | 4.24           | 7.58        | 3.15       | MO        | MO      |
| buffer_unsched_huge          | TO           | 5.67           | 16.54       | 3.43       | TO        | MO      |
| buffer_unsched_small         | TO           | 2.5            | 4.51        | 2.33       | MO        | 740.58  |
| buffer_unsched_tiny          | TO           | 2.4            | 3.27        | 1.69       | 3.76      | 77.41   |
| den_large                    | TO           | TO             | TO          | MO         | TO        | TO      |
| den_medium                   | TO           | 1364.3         | 831.79      | 272.32     | MO        | MO      |
| den_small                    | 4989.81      | 50.64          | 44.16       | 15.24      | 1256.58   | TO      |
| gni_lmcs.p1_2                | 1.65         | 1.46           | 1.86        | 2.18       | 2.32      | 16.59   |
| gni_lmcs.p1_2_1bit           | 2.01         | 1.94           | 2.84        | 1.7        | 1.8       | 15.76   |
| gni_p1_2                     | 3.7          | TO             | 3.23        | TO         | 1.73      | 2.62    |
| gni_p1_2_prop_1_bit          | 5.29         | TO             | TO          | TO         | 1.97      | 1.67    |
| gni_p1_2_sync                | 1.64         | 1.64           | 2.06        | 1.65       | 4.06      | MO      |
| gni_p1_4                     | 4.47         | TO             | 3.57        | TO         | 5.35      | 26.84   |
| gni_p1_4_prop_1_bit          | 16.04        | TO             | TO          | TO         | 2.54      | 18.97   |
| gni_p1_4_sync                | 2.44         | 2.78           | 2.94        | 1.86       | MO        | TO      |
| gni_p1_6                     | 7.9          | TO             | 2.92        | TO         | TO        | MO      |
| gni_p1_6_prop_1_bit          | 27.16        | TO             | TO          | TO         | TO        | 1398.39 |
| gni_p1_6_sync                | 2.53         | 2.7            | 2.17        | 1.87       | TO        | MO      |
| gni_p2_2                     | 30.67        | TO             | 5.22        | TO         | TO        | 1.36    |
| gni_p2_2_prop_1_bit          | 175.14       | TO             | TO          | TO         | TO        | 1.49    |
| gni_p2_2_sync                | 2.61         | 2.19           | 2.81        | 2.47       | TO        | MO      |
| gni_p2_4                     | 82.89        | TO             | 10.78       | TO         | MO        | MO      |
| gni_p2_4_prop_1_bit          | TO           | TO             | TO          | TO         | MO        | MO      |
| gni_p2_4_sync                | 9.47         | 4.78           | 5.27        | 2.46       | MO        | MO      |
| gni_p2_6                     | 61.88        | 2.34           | 3.73        | 2.45       | TO        | MO      |
| gni_p2_6_prop_1_bit          | TO           | TO             | TO          | MO         | TO        | MO      |
| gni_p2_6_sync                | 11.14        | 3.14           | 3.51        | 2.92       | TO        | MO      |
| gni_p3_2                     | 4.18         | TO             | 6.73        | TO         | 1.65      | 3.91    |

Table 4: Table comparing virtual best of each configuration (part 1)

| Instance                         | vb-induction | vb-kind-no-ref | vb-kind-ref | k-ind-incr | AutoHyper | HyPro  |
|----------------------------------|--------------|----------------|-------------|------------|-----------|--------|
| gni_p3_2_prop_1_bit              | 6.49         | TO             | TO          | TO         | 2.09      | 3.77   |
| gni_p3_4                         | 65.94        | TO             | 5.64        | TO         | TO        | 166.39 |
| gni_p3_4_prop_1_bit              | 18.67        | TO             | TO          | TO         | TO        | 91.95  |
| gni_p3_6                         | 609.3        | TO             | 12.29       | TO         | TO        | MO     |
| gni_p3_6_prop_1_bit              | 18.7         | TO             | TO          | TO         | TO        | 104.41 |
| gni_p4_2                         | 5.0          | TO             | 1532.42     | TO         | 1.36      | 4.49   |
| gni_p4_2_prop_1_bit              | 5.57         | TO             | TO          | TO         | 2.0       | 4.16   |
| gni_p4_2_sync                    | 2.36         | TO             | 3.4         | TO         | TO        | TO     |
| gni_p4_4                         | 12.31        | TO             | 3070.41     | TO         | MO        | 164.19 |
| gni_p4_4_prop_1_bit              | 9.65         | TO             | TO          | TO         | MO        | 131.36 |
| gni_p4_4_sync                    | 3.42         | TO             | 3.8         | TO         | MO        | TO     |
| gni_p4_6                         | 34.03        | TO             | TO          | TO         | TO        | MO     |
| gni_p4_6_prop_1_bit              | 29.86        | TO             | TO          | TO         | TO        | MO     |
| gni_p4_6_sync                    | 5.79         | TO             | 2.85        | TO         | TO        | MO     |
| lmcs_gni_p1_2                    | 1.74         | 1.35           | 1.84        | 2.23       | 1.46      | 1.98   |
| lmcs_gni_p1_2_prop_1_bit         | 2.56         | 2.34           | 2.97        | 2.06       | 1.67      | 1.9    |
| lmcs_gni_p1_4                    | 2.46         | 1.76           | 3.02        | 1.67       | 1.94      | 1.56   |
| lmcs_gni_p1_4_prop_1_bit         | 2.35         | 2.59           | 3.34        | 2.79       | 1.62      | 1.51   |
| lmcs_gni_p1_6                    | 2.7          | 2.03           | 2.42        | 1.8        | 11.8      | 11.24  |
| lmcs_gni_p1_6_prop_1_bit         | 3.97         | 2.91           | 3.95        | 2.22       | 1.5       | 1.53   |
| lmcs_gni_p2_2                    | 3.31         | 2.5            | 2.24        | 2.01       | 1.64      | 9.89   |
| lmcs_gni_p2_2_prop_1_bit         | 5.36         | TO             | 34.21       | TO         | 2.24      | 11.13  |
| lmcs_gni_p2_4                    | 6.98         | TO             | 3.32        | TO         | MO        | MO     |
| lmcs_gni_p2_4_prop_1_bit         | 11.78        | TO             | 57.6        | TO         | 3.0       | MO     |
| lmcs_gni_p2_6                    | 8.47         | 2.34           | 2.98        | 2.66       | TO        | MO     |
| lmcs_gni_p2_6_prop_1_bit         | 28.28        | TO             | 97.62       | TO         | TO        | MO     |
| lmcs_gni_p3_2                    | 3.82         | TO             | TO          | TO         | 1.78      | 80.05  |
| lmcs_gni_p3_2_prop_1_bit         | 5.46         | TO             | TO          | TO         | 2.29      | 92.32  |
| lmcs_gni_p3_4                    | 19.23        | 2.14           | 3.06        | 2.63       | 15.08     | TO     |
| lmcs_gni_p3_4_prop_1_bit         | 11.03        | TO             | TO          | TO         | MO        | TO     |
| lmcs_gni_p3_6                    | 2.72         | 2.26           | 2.85        | 2.45       | TO        | MO     |
| lmcs_gni_p3_6_prop_1_bit         | 12.87        | TO             | TO          | TO         | MO        | TO     |
| lmcs_gni_p4_2                    | 31.03        | 2.16           | 2.79        | 1.84       | 1.72      | MO     |
| lmcs_gni_p4_2_prop_1_bit         | 42.98        | TO             | TO          | TO         | 1.88      | MO     |
| lmcs_gni_p4_4                    | 16.74        | 2.19           | 3.22        | 1.55       | 14.35     | TO     |
| lmcs_gni_p4_4_prop_1_bit         | 72.18        | TO             | TO          | TO         | MO        | TO     |
| lmcs_gni_p4_6                    | 53.97        | 2.63           | 3.66        | 2.41       | TO        | TO     |
| lmcs_gni_p4_6_prop_1_bit         | 105.69       | TO             | TO          | TO         | TO        | TO     |
| mutation                         | 1.2          | 1.61           | 1.86        | 2.03       | 2.48      | UNK    |
| robot_robust_100                 | TO           | 12.13          | 26.3        | 5.21       | 3.4       | UNK    |
| robot_robust_1600                | TO           | 47.26          | 418.69      | 20.5       | 4.78      | UNK    |
| robot_robust_3600                | TO           | 76.43          | 941.64      | 29.46      | 2585.88   | UNK    |
| robot_sp_100                     | TO           | 31.42          | 51.23       | 12.99      | 1.61      | UNK    |
| robot_sp_1600                    | TO           | 1180.04        | 2156.88     | 580.02     | 6.83      | UNK    |
| robot_sp_3600                    | TO           | 5492.98        | TO          | MO         | 17.51     | UNK    |
| simple_program_1                 | 2.1          | 4.88           | 2.2         | 1.71       | 1.53      | 1.82   |
| simple_program_2                 | 2.37         | 7.32           | 2.94        | 2.16       | 1.9       | 2.65   |
| simple_program_red_locs_small_p1 | 5.49         | 95.31          | 7.38        | 4.05       | 1.26      | 3.66   |
| snark                            | TO           | 13.71          | 73.03       | 10.58      | 23 115.78 | MO     |

Table 5: Table comparing virtual best of each configuration (part 2)